Bitácora de Sesión — 2026-02-18

Paninos Digital · Sprint de Checkout, Domicilios y UX

Contexto de la sesión

Fecha: 18 de febrero de 2026
Duración aproximada: ~2 horas
Rama: `main`
Commit de cierre: `9173caa` — `feat: domicilios identificación de local por barrio`
Archivos modificados: 8 archivos · 907 líneas añadidas

Esta sesión se enfocó en tres áreas principales:

1. Correcciones de bugs en el flujo de upsell de bebidas
2. Sección de método de pago en el checkout
3. Implementación completa del flujo de domicilios con verificación de cobertura por barrio

Bugs corregidos

🔘 Bug: `<button>` anidado en `UpsellDrinksModal`

Síntoma: Error en consola `In HTML, <button> cannot be a descendant of <button>. This will cause a hydration error.`

Causa: la tarjeta de bebida era un `<button>` (para seleccionar) que contenía otro `<button>` (el ✕ para quitar). HTML no permite botones anidados.

Solución: La tarjeta se convirtió en `<div role="button">` con `onClick` + `onKeyDown` para mantener accesibilidad. El botón ✕ quedó como hermano posicionado en absoluto, no como hijo.

```
// ❌ Antes — HTML inválido
<button onClick={increment}>
    <button onClick={decrement}>✕</button>
</button>

// ☑ Ahora — estructura correcta
<div role="button" onClick={increment}>
    ...
</div>
<button onClick={(e) => { e.stopPropagation(); decrement(); }}>✕</button>
```

Nuevas funcionalidades

1. Modal de Upsell de Bebidas (`UpsellDrinksModal.jsx`)

Propósito: Aumentar el ticket promedio ofreciendo bebidas cuando el cliente solo tiene sandwiches en el carrito.

Lógica:

- Se activa desde `CartDrawer` al tocar "Finalizar Pedido"
- Solo aparece si el carrito contiene **únicamente sandwiches** (sin bebidas ya agregadas)
- Si ya hay bebidas → va directo al checkout sin modal

UX:

- Bottom sheet en móvil, modal centrado en desktop
- Grid de 5 bebidas: Agua · Gaseosa · Jugo Valle · Fuze Tea · Coronita
- Cada toque incrementa la cantidad (máx 5 unidades)
- Botón ✕ para quitar una bebida
- Resumen dinámico: `"2 bebidas seleccionadas · +$7.000"`
- CTA primario: "Agregar y finalizar pedido" / "Continuar sin bebida"

Estructura lista para imágenes reales:

```
/public/images/drinks/
├── agua.png
├── gaseosa.png
├── jugo.png
├── fuzetea.png
└── coronita.png
```

Cuando se suban estos archivos, aparecen automáticamente. Mientras tanto muestra un placeholder SVG.

---

2. Método de Pago en Checkout (`Checkout.jsx`)

Propósito: Dar al cliente la opción de pagar en tienda o en línea.

Opciones implementadas:

Método	Descripción	Estado
Pagar en tienda	Efectivo o datáfono al recoger	☑ Activo (default)
Pagar en línea (Bold)	Débito y crédito, sin salir de la app	☐ Pendiente integración Bold

Comportamiento:

- Radio visual con animación al seleccionar
- Al elegir Bold → aparece info adicional: "*Bold acepta todas las tarjetas Visa, Mastercard y American Express*"
- El botón de confirmar cambia de texto: `"Confirmar Pedido"` (tienda) vs `"Pagar"` (online)
- El campo `payment_method` se incluye en el payload del pedido y en la URL de confirmación

---

3. Pantalla de Confirmación mejorada (`confirmacion/page.js`)

Cambios:

- **Logo:** reemplazado `<Image src="/images/logo.png" />` (invisible en fondo oscuro) por badge de texto amarillo `PANINOS` — siempre visible
- **Método de pago:** aparece bajo el número de pedido con ícono: "*Pago en tienda al recoger*" o "*Pago en línea · Bold*"
- **Emojis eliminados:** los 3 pasos (Prepara, WhatsApp, Recoge) ahora usan íconos SVG en contenedor amarillo
- **Saludo:** eliminado el emoji 👀 para una apariencia más profesional

---

4. Página "Sin Cobertura" (`NoCobertura.jsx`)

Propósito: Mostrar cuando la dirección del cliente no tiene cobertura de domicilio.

Elementos:

- Ícono de pin de mapa con badge rojo de ✕
- Muestra la dirección/barrio que intentó el cliente
- Lista de barrios destacados por sede (para que el cliente sepa dónde sí cubrimos)
- CTA primario: **"Recoger en Tienda"** → regresa al Home
- CTA secundario: **"Ingresar otra dirección"** → regresa al input de barrio
- Enlace de ayuda por WhatsApp en el footer
- Es un componente reutilizable con props `address`, `onTryAgain`, `onPickup`

---

5. Flujo Completo de Domicilios (`DeliveryFlow.jsx`)

Propósito: Permitir que el cliente ingrese su barrio y verificar si tiene cobertura antes de mostrar el menú.

Flujo:

```
Home → PEDIR A DOMICILIO
    → DeliveryFlow (pantalla completa)
        → Input de barrio + autocomplete
        → checkCoverage()
            ├── Con cobertura → asigna sede → /menu?type=delivery
            └── Sin cobertura → NoCobertura
                    ├── Recoger en Tienda → Home
                    └── Ingresar otra dirección → vuelve al input
```

Autocomplete:

- Busca en `ALL_CALI_NEIGHBORHOODS` (~250 barrios)
- Cada sugerencia muestra indicador visual: ◍ con cobertura / ◯ sin cobertura
- Solo requiere 2 caracteres para activarse
- Máximo 8 sugerencias

Puntos de acceso:

1. Botón "PEDIR A DOMICILIO" en el Hero de la Home
2. "Prefiero pedir a domicilio" dentro del `LocationSelector` (selector de pickup)

---

6. Base de datos de zonas de cobertura (`lib/deliveryZones.js`)

Propósito: Fuente de verdad para las zonas de delivery. Sin dependencia de Google Maps.

Arquitectura (2 capas):

Capa	Descripción	Tamaño
`ALL_CALI_NEIGHBORHOODS`	Todos los barrios de Cali (autocomplete)	~250 barrios
`COBERTURA_LIBERTADORES`	Barrios con entrega desde Sede Libertadores	41 barrios
`COBERTURA_CALDAS`	Barrios con entrega desde Sede Caldas	27 barrios
Total con cobertura		**68 barrios**

Zonas basadas en los mapas del cliente:

Sede	Zona geográfica
Libertadores	Centro-norte: Chipichape → Huandama, Cras 15–70
Caldas	Sur-occidente: El Guabal → ICESI/Pance, Cras 28–100+

Funciones exportadas:

- `checkCoverage(neighborhood)` → `{ covered, sede_slug }` o `{ covered: false }`
- `getNeighborhoodSuggestions(query, limit)` → array con `{ name, covered, sede }`
- `COVERAGE_ZONES_DISPLAY` → datos para mostrar en UI
- `TOTAL_COVERAGE_BARRIOS` → conteo total

Para agregar barrios:

```
// En COBERTURA_CALDAS o COBERTURA_LIBERTADORES (minúsculas, sin tilde):
const COBERTURA_CALDAS = new Set([
    ...
    'nuevo barrio',  // ← agregar aquí
]);
// Si el barrio no existe en ALL_CALI_NEIGHBORHOODS, agregarlo allí también.
```

Archivos modificados / creados

Archivo	Tipo	Descripción
`frontend/components/UpsellDrinksModal.jsx`	🆕 Nuevo	Modal upsell de bebidas

Archivo	Tipo	Descripción
`frontend/components/CartDrawer.jsx`	✏️ Modificado	Activa upsell antes del checkout
`frontend/components/Checkout.jsx`	✏️ Modificado	Sección de método de pago
`frontend/app/confirmacion/page.js`	✏️ Modificado	Logo visible, SVGs, método de pago
`frontend/components/NoCobertura.jsx`	🆕 Nuevo	Pantalla sin cobertura
`frontend/components/DeliveryFlow.jsx`	🆕 Nuevo	Flujo completo domicilio + barrio
`frontend/lib/deliveryZones.js`	🆕 Nuevo	Base de datos barrios + cobertura
`frontend/components/Home.jsx`	✏️ Modificado	Botón domicilio → DeliveryFlow
`frontend/components/LocationSelector.jsx`	✏️ Modificado	"Pedir a domicilio" → DeliveryFlow

Estado del backend

No se realizaron cambios en backend esta sesión.

- No hay migraciones nuevas
- No hay nuevos endpoints
- El endpoint de órdenes (`POST /api/orders/`) sigue **simulado** en el frontend

```
// Checkout.jsx — la llamada real sigue comentada:
// const response = await api.post('orders/', orderPayload);  ← pendiente Sprint 3
await new Promise(resolve => setTimeout(resolve, 1500));
const response = { data: { id: `PAN-${Date.now()}` } };
```

Despliegue a producción

Solo reconstruir el frontend:

```
# En el servidor VPS
git pull origin main
docker compose -f docker-compose.prod.yml up -d --build --force-recreate frontend
docker logs paninos_frontend --tail 30
```

No se requiere:

- `python manage.py migrate` (sin cambios de modelo)

- Reconstruir el backend (sin cambios en Django)

Pendientes para próximas sesiones

- ☐ **Imágenes de bebidas:** subir archivos a `/public/images/drinks/` (agua, gaseosa, jugo, fuzetea, coronita)
- ☐ **Integración Bold:** conectar el flujo de pago en línea con el gateway Bold
- ☐ **Endpoint de órdenes:** implementar `POST /api/orders/` en Django + integración Loggro
- ☐ **Notificación WhatsApp:** enviar mensaje al cliente cuando el pedido esté confirmado
- ☐ **Google Maps (Fase 2):** reemplazar el sistema de barrios por polígonos geográficos exactos para mayor precisión
- ☐ **Panel de administración:** vista para que el equipo de Paninos vea pedidos entrantes en tiempo real
- ☐ **Ampliar zonas:** confirmar con el cliente qué barrios adicionales agregar a `deliveryZones.js`

Documento generado automáticamente al cierre de sesión · Paninos Digital · 2026-02-18